

Data Architecture and Storage Pipeline

Filesystem Design, Tensor Representation, and Anomaly Repair
for the GBT+RL 0DTE Options Trading System

Kripta Studios Quantitative Research Division

March 2026

Contents

1	Introduction	2
2	Filesystem Architecture	2
2.1	Primary Historical Store: <code>data_options</code>	2
2.2	Derived Spot Price Store: <code>data_underlying_derived</code>	3
2.3	Real-Time Feed: <code>rt_data</code>	3
3	Design Matrix Construction	4
3.1	Flat Feature Matrix (GBT Training)	4
3.2	RL Options Cache	4
3.3	Episode Index	5
4	Chunked LRU Cache for RL Training	5
4.1	Memory Constraint	5
4.2	Parallel Preprocessing	6
5	Data Anomaly Repair	6
5.1	Zero-Value Anomalies at Market Open	6
5.2	Residual Gap Imputation	6
5.3	Feature-Level Numerical Stability	7
6	Data Volume and Retention Policy	7

1 Introduction

This document describes the complete data storage architecture, ingestion pipeline, and tensor feature structures underpinning the GBT+RL 0DTE options trading system. The infrastructure is designed to serve three distinct consumers: (1) the GBT walk-forward training pipeline, which demands a flat `.parquet` design matrix $\mathbf{X} \in \mathbb{R}^{N \times D}$ with $D = 163$ features per 5-minute bar; (2) the PPO reinforcement learning environment, which requires per-episode access to real-time options chains at nanosecond-resolved timestamps; and (3) the live trading bot, which consumes a rolling real-time feed updated at 60-second intervals.

All three consumers share identical algebraic feature mappings, defined in `compute_features.py` and `realtime_feed.py`, guaranteeing strict model parity between training-time and inference-time state representations.

The data sourced in this system is the historical and live options data obtained from the ThetaData API (v3), which provides end-of-minute OHLC bars, implied volatility, full Greeks ($\Delta, \Gamma, \theta, \mathcal{V}, \rho$), and open interest for all listed SPX/SPXW/SPY/QQQ strikes from 2024 onwards.

2 Filesystem Architecture

2.1 Primary Historical Store: `data_options`

The foundational data ingestion from the ThetaData API is partitioned into daily `.parquet` files compressed with `snappy`, which provides near-optimal read throughput for columnar scans while maintaining acceptable write latency during bulk download. The directory hierarchy is organized as:

```
D:/ThetaData/data_options/  
{SYMBOL}/ # SPXW, SPX, SPY, QQQ, VIX, TLT  
  greeks/  
    {YEAR}/  
      {MONTH}/  
        {SYM}_{expiry}_{trade_date}_greeks.parquet  
  ohlc/  
    {YEAR}/{MONTH}/  
      {SYM}_{expiry}_{trade_date}_ohlc.parquet  
  oi/  
    ...  
  iv/  
    ...
```

The canonical filename convention encodes both the option's expiration date T and the active trading date t :

$$\text{filename} ::= \{\text{Symbol}\}_{\{T\}}_{\{t\}}_{\{\text{type}\}}.\text{parquet} \quad (1)$$

This dual-date convention allows the system to efficiently load all contracts with $T = t$ (0DTE in-

struments) as well as weekly or multi-day expirations by filtering on the expiry field. The files are discovered via a recursive directory scan in `options_downloader.py` and `run_preprocess.py`.

Definition 1 (0DTE Contract). *A 0DTE (zero days to expiration) contract is any options contract for which the expiration date T is equal to the current trading date t : $T = t$. For SPX, these are SPXW contracts listed under the CBOE’s Friday-daily listing program, available every trading day since 2022.*

2.2 Derived Spot Price Store: `data_underlying_derived`

Because the CBOE imposes endpoint restrictions on raw SPX index historical data via most retail data vendors, the pipeline reconstructs spot price S_t implicitly from the `underlying_price` field embedded within the options Greeks metadata provided by ThetaData. This field reports the settlement spot price used by the ThetaData matching engine at the time of the quote.

The derived spot price at time t is estimated as the mean of the `underlying_price` field across all listed strikes $\mathcal{K}_t$ for the nearest 0DTE expiration:

$$\hat{S}_t = \frac{1}{|\mathcal{K}_t|} \sum_{k \in \mathcal{K}_t} u_{t,k,T} \quad (2)$$

where $u_{t,k,T}$ denotes the `underlying_price` field for strike k , expiry T , at time t . This derivation guarantees synchronization between the options pricing surface and the spot reference frame, eliminating latency asymmetries present in disparate data feeds.

The `underlying_downloader.py` script writes these derived spot prices to:

```
D:/ThetaData/data_underlying_derived/
{SYMBOL}/
  {YEAR}/{MONTH}/
    {SYM}_{trade_date}_spot.parquet
```

2.3 Real-Time Feed: `rt_data`

The live trading infrastructure maintains a rolling real-time store, updated every 60 seconds by the `realtime_feed.py` microservice:

```
rt_data/
{YYYYMMDD}/
  SPXW_greeks_0dte_latest.parquet
  SPXW_oi_weekly_latest.parquet
  spot_SPX_latest.parquet
  spot_VIX_latest.parquet
  spot_TLT_latest.parquet
```

On service startup, the feed downloads backfill data for trading days $D-1$ through $D-5$, ensuring that all multi-day Initial Balance features (Section ??) are populated from the first active minute bar at 09:30 ET. This prevents cold-start failures in the Initial Balance computation and multi-day confluence features.

The asynchronous polling daemon targets the ThetaData v3 API via:

$$\text{http://127.0.0.1:25503/v3} \quad (\text{ThetaTerminal local proxy}) \quad (3)$$

with all HTTP calls routed through `httpx.AsyncClient`. The polling interval is configurable via `-interval` (default: 60 seconds), and the pipeline achieves end-to-end latency of $\mathcal{O}(150 \text{ ms})$ from market tick to persisted `.parquet` write.

3 Design Matrix Construction

3.1 Flat Feature Matrix (GBT Training)

The primary artifact consumed by the GBT classifier and RL episode-generation pipeline is a flat Parquet design matrix:

$$\mathbf{X} \in \mathbb{R}^{N \times D}, \quad D = 163, \quad N \approx 138,000 \quad (4)$$

Each row corresponds to a single 5-minute bar for one of the tracked underlyings (SPX, SPY, QQQ). The matrix is constructed by `collect_training_data_parquet.py` using W parallel workers (configurable via `-workers`), with each worker independently processing a date partition.

The target label $y_t \in \{0, 1, 2\}$ is assigned by the labeling oracle:

$$y_t = \begin{cases} 2 \text{ (LONG)} & \text{if } \max_{\tau \in (t, t+T_{\max}]} S_{\tau} \geq S_t(1 + \theta_L) \text{ before } S_{\tau} \leq S_t(1 - \theta_S) \\ 0 \text{ (SHORT)} & \text{if } \min_{\tau \in (t, t+T_{\max}]} S_{\tau} \leq S_t(1 - \theta_S) \text{ before } S_{\tau} \geq S_t(1 + \theta_L) \\ 1 \text{ (HOLD)} & \text{otherwise} \end{cases} \quad (5)$$

where $\theta_L = 1.0\%$, $\theta_S = 0.5\%$, and $T_{\max} = 180$ minutes are the default long target, stop threshold, and lookahead horizon respectively.

Remark 1. *The class imbalance in $\mathbf{X}$ is severe: directional signals (LONG or SHORT) constitute approximately 13% of all bars. The walk-forward training procedure (Section 3 of the Architecture document) addresses this via explicit class balancing, equalizing SHORT and LONG samples and capping HOLD at $2\times$ the total directional count per training window.*

3.2 RL Options Cache

The reinforcement learning environment requires access to real options chain data indexed by (t, T, K) tuples at each timestep. Loading the full historical options store into RAM is infeasible given typical archive sizes of ~ 80 GB. The `run_preprocess.py` script constructs a per-day serialized cache:

```
rl_data/
episode_index.parquet # N_ep x D episode index
rl_options_cache_chunks/
    20240102.pkl
```

20240103.pkl

...

Each YYYYMMDD.pkl shard contains a Python dictionary keyed by "YYYYMMDD_HH:MM" timestamps, mapping to a list of option records with fields: `strike`, `delta`, `gamma`, `theta`, `vega`, `bid`, `ask`, `mid`, `open_interest`.

3.3 Episode Index

The episode index is a Parquet file with one row per candidate trade signal:

$$\mathbf{E} \in \mathbb{R}^{N_{\text{ep}} \times (D+4)} \quad (6)$$

Each row contains the full 163-dimensional feature vector at the signal timestamp, plus derived GBT prediction columns: `mlp_direction` $\in \{\text{SHORT}, \text{LONG}\}$, `mlp_confidence` $\in [0, 1]$, `mlp_time_to_target` (normalized 0–1), and `episode_id`.

Only rows with `mlp_direction` $\neq \text{HOLD}$ and `mlp_confidence` ≥ 0.55 are retained, yielding $N_{\text{ep}} \ll N$ episodes. The RL training curriculum then further filters these by confidence tier (Section 5 of the RL Model document).

4 Chunked LRU Cache for RL Training

4.1 Memory Constraint

Loading all daily options shards simultaneously would require $\mathcal{O}(G_{\text{total}})$ RAM, where G_{total} denotes the total uncompressed size of the options archive. To bound memory consumption, the `ChunkedOptionsCache` class implements a Least Recently Used (LRU) eviction policy parameterized by `max_days_in_ram = k`:

Definition 2 (`ChunkedOptionsCache`). *Let $\mathcal{D} = \{d_1, d_2, \dots, d_M\}$ be the set of all available trading day shards. The `ChunkedOptionsCache` maintains an in-memory dictionary $\mathcal{L} \subseteq \mathcal{D}$ with $|\mathcal{L}| \leq k$. On a cache miss for day d , the shard d is loaded from disk into $\mathcal{L}$; if $|\mathcal{L}| > k$, the least recently accessed day is evicted:*

$$d_{\text{evict}} = \arg \max_{d \in \mathcal{L}} (t_{\text{now}} - t_{\text{access}}(d)) \quad (7)$$

The key lookup interface is:

$$\text{cache.get}(\underbrace{\text{"YYYYMMDD_HH:MM"}}_{\text{composite key}}) \rightarrow \text{option records} \mid \text{None} \quad (8)$$

The date prefix YYYYMMDD is extracted from the key string to determine which daily shard to load. This design reduces RAM usage from $\mathcal{O}(M)$ to $\mathcal{O}(k)$ days, where $k = 30\text{--}120$ is configurable via `-max-days-in-ram`.

4.2 Parallel Preprocessing

The preprocessing pipeline (`run_preprocess.py`) constructs the shards using W parallel workers (default $W = 24$), achieving near-linear speedup across date partitions:

$$T_{\text{preprocess}} \approx \frac{M \cdot \bar{t}_{\text{day}}}{W} \quad (9)$$

where $\bar{t}_{\text{day}}$ is the average per-day processing time. For a 2-year historical archive ($M \approx 500$ trading days), this yields preprocessing times of approximately 15–40 minutes on a 24-core workstation.

5 Data Anomaly Repair

5.1 Zero-Value Anomalies at Market Open

ThetaData exhibits a systematic data quality issue at the 09:30:00 ET open bar: OHLC fields occasionally report zero values due to quote latency in the first minute of trading. This manifests as $S_{09:30} = 0$ in the underlying OHLC, which would propagate catastrophic errors through all downstream features.

The `data_corrector.py` script (`services/`) applies a cascading repair procedure. Let $x_t^{(f)}$ denote field $f \in \{\text{open, high, low, close}\}$ at the open bar $t = 09 : 30$:

$$\hat{x}_t^{(f)} = \begin{cases} x_t^{\text{close}} & \text{if } x_t^{(f)} = 0 \wedge x_t^{\text{close}} > 0 \\ \text{NaN} & \text{if } x_t^{(f)} = 0 \wedge x_t^{\text{close}} = 0 \\ x_t^{(f)} & \text{otherwise} \end{cases} \quad (10)$$

5.2 Residual Gap Imputation

After zero repair, any residual NaN values resulting from missing minute bars (e.g., early-close sessions, system outages) are imputed via a two-pass temporal propagation:

$$\hat{x}_t = \begin{cases} x_{t+1} & \text{backward fill: propagate next valid value} \\ x_{t-1} & \text{forward fill: propagate last valid value (fallback)} \end{cases} \quad (11)$$

This sequential `bfill` $\rightarrow$ `ffill` scheme ensures that no NaN values survive into the feature computation stage. The corrector operates on the `data_underlying_derived` directory and is invoked on demand via:

```
python services/data_corrector.py \  
--data-dir D:/ThetaData/data_underlying_derived \  
--symbols SPXW SPY QQQ VIX TLT
```

5.3 Feature-Level Numerical Stability

Several derived features involve ratios of Greek quantities (e.g., `delta_gamma_ratio`, `net_zomma`) that can assume values exceeding 10^6 in magnitude during low-gamma regimes. These are regularized via a sign-preserving log transform applied during data collection:

$$\tilde{x} = \text{sign}(x) \cdot \log(1 + |x|) \quad (12)$$

This transform is applied before the feature matrix is written to Parquet. It reduces the kurtosis of extreme-valued features from $\sim 10^3$ to $\mathcal{O}(10)$, improving both GBT split quality and gradient stability during RL training. The untransformed values are never stored in the training Parquet.

At inference time in the GBT normalizer (`FeatureNormalizer.transform`), a further standardization is applied to center and scale each feature to approximate zero mean and unit variance, using statistics computed from the training window:

$$x_{\text{norm}} = \frac{\tilde{x} - \mu_{\text{train}}}{\sigma_{\text{train}} + \varepsilon} \quad (13)$$

with $\varepsilon = 10^{-8}$ for numerical stability.

6 Data Volume and Retention Policy

Component	Raw Size	Compressed	Retention
<code>data_options</code> (SPXW/SPX/SPY/QQQ)	~ 80 GB	28 GB	Permanent
<code>data_underlying_derived</code>	~ 2 GB	600 MB	Permanent
<code>training_data_derived.parquet</code>	2–4 GB	600 MB	Per-run
<code>rl_options_cache_chunks</code>	~ 8 GB	N/A	Per-run
<code>rt_data</code> (rolling 5 days)	~ 50 MB	15 MB	Rolling

Table 1: Approximate storage requirements by pipeline component. Compressed sizes assume `snappy` encoding. “Per-run” artifacts can be regenerated from raw data.

All `.parquet` files use `pyarrow` as the serialization engine with the `snappy` codec. This provides columnar read latency of approximately 100–300 ms for a single day’s options chain on NVMe storage, enabling the RL environment to load missing day shards within the episode reset timeout.

References

- [1] ThetaData Systems, “ThetaData API v3 Documentation,” <https://docs.thetadata.us>, 2024.
- [2] Apache Software Foundation, “Apache Arrow: Columnar In-Memory Analytics,” <https://arrow.apache.org>, 2024.

- [3] G. Ke et al., “LightGBM: A Highly Efficient Gradient Boosting Decision Tree,” *NeurIPS*, vol. 30, 2017.
- [4] J. Schulman et al., “Proximal Policy Optimization Algorithms,” *arXiv:1707.06347*, 2017.